

SEMINARARBEIT

Im Studiengang FH Technikum Wien, Bachelorstudiengang BIF
Lehrveranstaltung BIF-VZ-5-WS2025-INFC-EN

Terraform Workshop Protocol

Ausgeführt von: David Veigel

Kevin Forter

Personenkennzeichen: XXXXXXXXXXXXXXXX

BegutachterIn:

Wien, den 3. Dezember 2025

Inhaltsverzeichnis

1	Workshop 1: Terraform Basics	1
1.1	Configuration	1
1.1.1	Provider and Region	1
1.1.2	VPC and Subnets	1
1.1.3	Security Group	1
1.1.4	EC2 Instance	2
1.1.5	Output and Verification	3
1.2	Execution	4
2	Workshop 2: Advanced Networking	5
2.1	Network Infrastructure	5
2.1.1	VPC and Internet Gateway	5
2.1.2	Subnet and Routing	7
2.2	Compute and Access	8
2.2.1	Security Group	8
2.2.2	Network Interface and Elastic IP	9
2.2.3	EC2 Instance	9
2.3	Execution	10
3	Workshop 3: Load Balancing and High Availability	11
3.1	Networking Setup	11
3.1.1	VPC and Internet Gateway	11
3.1.2	Subnets in Multiple AZs	12
3.1.3	Routing	14
3.2	Security Groups	15
3.2.1	Load Balancer Security Group	15
3.2.2	EC2 Security Group	15
3.3	Load Balancer Configuration	16
3.3.1	Application Load Balancer	16
3.3.2	Target Group and Listener	17
3.4	Compute Resources	18
3.4.1	EC2 Instances	18
3.4.2	Target Group Attachment	20
3.5	Verification	20

3.6	Automation and CI/CD	21
3.6.1	GitHub Configuration	21
3.6.2	Terraform Cloud Configuration	22
3.6.3	CI/CD Pipeline	22

1 Workshop 1: Terraform Basics

In this workshop, we set up a simple web server on AWS using Terraform. The goal is to provision an EC2 instance in the default VPC, install an Apache web server using user data, and verify that it is reachable.

1.1 Configuration

1.1.1 Provider and Region

We configure the AWS provider to use the `us-east-1` region, as specified in the requirements.

```
provider "aws" {  
    region = "us-east-1"  
}
```

1.1.2 VPC and Subnets

Instead of creating a new network, we use the existing default VPC and its subnets. This simplifies the setup for this initial workshop.

```
data "aws_vpc" "default" {  
    default = true  
}  
  
data "aws_subnets" "default_vpc_subnets" {  
    filter {  
        name     = "vpc-id"  
        values = [data.aws_vpc.default.id]  
    }  
}
```

1.1.3 Security Group

To make our web server accessible, we create a security group. We allow inbound TCP traffic on port 80 (HTTP) from any IP address (`0.0.0.0/0`). We also allow all outbound traffic to ensure the server can download updates and packages.

```

resource "aws_security_group" "web_sg" {
  name          = "tf-web-sg"
  description   = "Allow_HTTP_in,_all_egress"
  vpc_id        = data.aws_vpc.default.id

  ingress {
    description      = "HTTP"
    from_port        = 80
    to_port          = 80
    protocol          = "tcp"
    cidr_blocks      = ["0.0.0.0/0"]
  }

  egress {
    description      = "All_egress"
    from_port        = 0
    to_port          = 0
    protocol          = "-1"
    cidr_blocks      = ["0.0.0.0/0"]
  }
}

```

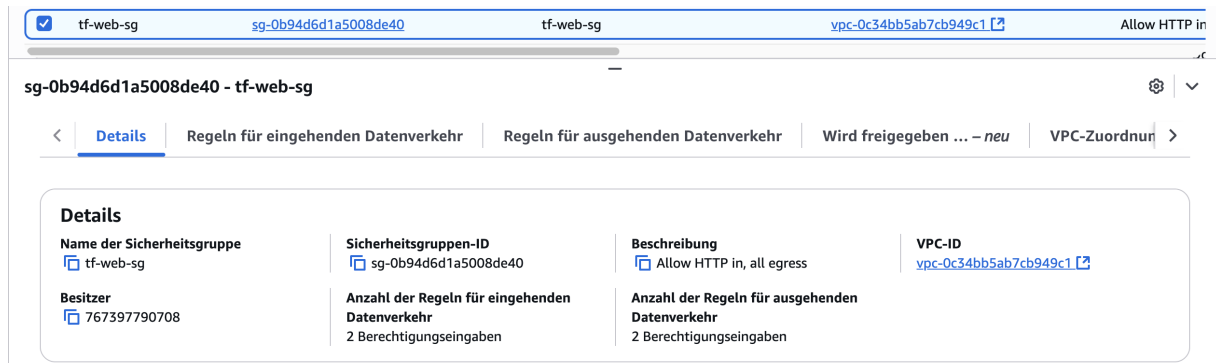


Abbildung 1: Security Group Configuration in AWS Console

1.1.4 EC2 Instance

We provision an EC2 instance using the `t2.micro` instance type, which is eligible for the free tier. We dynamically fetch the latest Ubuntu 22.04 LTS AMI. The `user_data` script is used to automatically install and start the Apache web server upon launch.

```

resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id    = data.aws_subnets.default_vpc_subnets.ids[0]
}

```

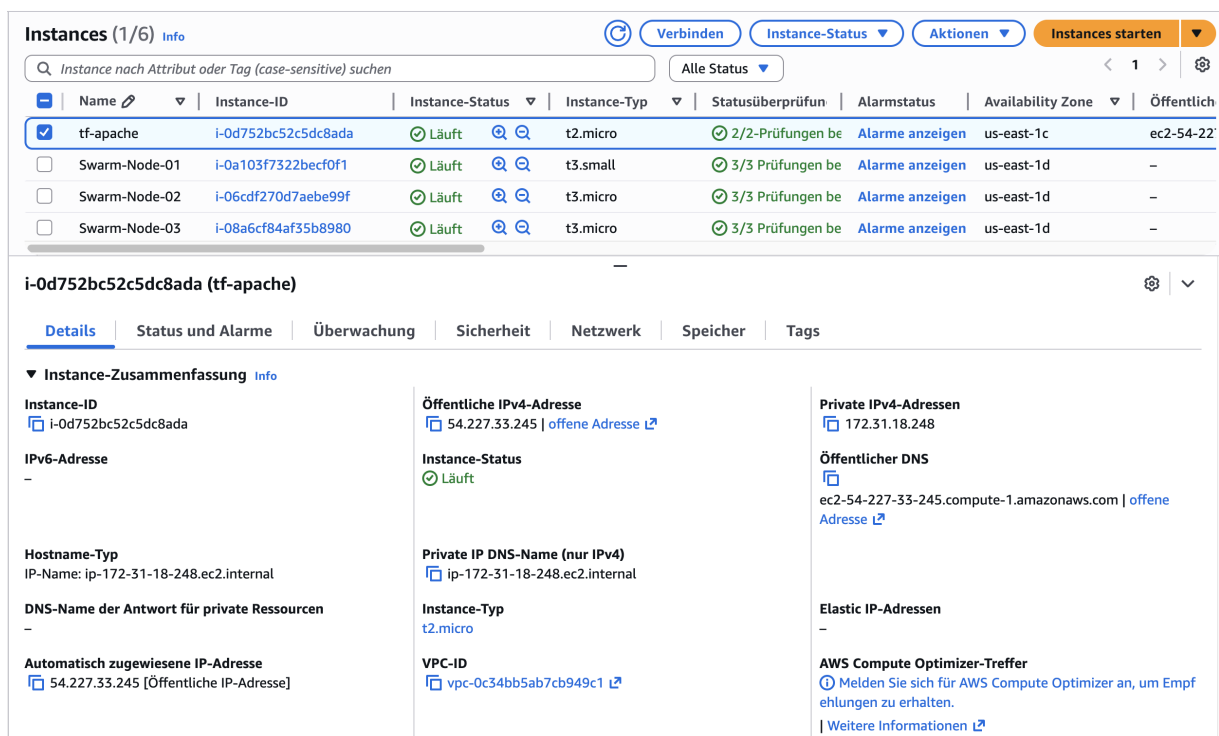
```

vpc_security_group_ids      = [aws_security_group.web_sg.id]
associate_public_ip_address = true

user_data = <<-EOF
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = {
  Name = "tf-apache"
}
}

```



Instances (1/6) Info

Q Instance nach Attribut oder Tag (case-sensitive) suchen

Alle Status

Name	Instance-ID	Instance-Status	Instance-Typ	Statusüberprüfun	Alarmstatus	Availability Zone	Öffentlich
☒ tf-apache	i-0d752bc52c5dc8ada	Lauft	t2.micro	2/2-Prüfungen be	Alarme anzeigen	us-east-1c	ec2-54-227
☐ Swarm-Node-01	i-0a103f7322becf0f1	Lauft	t3.small	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-
☐ Swarm-Node-02	i-06cdf270d7aeb99f	Lauft	t3.micro	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-
☐ Swarm-Node-03	i-08a6cf84af35b8980	Lauft	t3.micro	3/3 Prüfungen be	Alarme anzeigen	us-east-1d	-

i-0d752bc52c5dc8ada (tf-apache)

Details | Status und Alarme | Überwachung | Sicherheit | Netzwerk | Speicher | Tags

▼ Instance-Zusammenfassung Info

Instance-ID i-0d752bc52c5dc8ada	Öffentliche IPv4-Adresse 54.227.33.245 offene Adresse	Private IPv4-Adressen 172.31.18.248
IPv6-Adresse -	Instance-Status Lauft	Öffentlicher DNS ec2-54-227-33-245.compute-1.amazonaws.com offene Adresse
Hostname-Typ IP-Name: ip-172-31-18-248.ec2.internal	Private IP DNS-Name (nur IPv4) ip-172-31-18-248.ec2.internal	Elastic IP-Adressen -
DNS-Name der Antwort für private Ressourcen -	Instance-Typ t2.micro	AWS Compute Optimizer-Treffer Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. Weitere Informationen
Automatisch zugewiesene IP-Adresse 54.227.33.245 [Öffentliche IP-Adresse]	VPC-ID vpc-0c34bb5ab7cb949c1	

Abbildung 2: EC2 Instance Running in AWS Console

1.1.5 Output and Verification

Finally, we output the public DNS of the instance to easily access it. We also include a `null_resource` to verify that the web server is actually reachable via HTTP.

```

output "instance_public_dns" {

```

```
description = "Public_DNS_of_the_EC2_instance"
value       = aws_instance.web.public_dns
}
```

1.2 Execution

We initialize the Terraform project to download the necessary providers.

```
> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching ">= 5.0.0"...
- Finding latest version of hashicorp/null...
- Installing hashicorp/aws v6.20.0...
- Installed hashicorp/aws v6.20.0 (signed by HashiCorp)
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Abbildung 3: Terraform Init

Then we apply the configuration to create the resources.

```

> terraform apply -auto-approve
data.aws_vpc.default: Reading...
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 4s [id=ami-0c398cb65a93047f2]
data.aws_vpc.default: Read complete after 7s [id=vpc-0c34bb5ab7cb949c1]
data.aws_subnets.default_vpc_subnets: Reading...
data.aws_subnets.default_vpc_subnets: Read complete after 1s [id=us-east-1]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                        = "ami-0c398cb65a93047f2"
  + arn                      = (known after apply)
  + associate_public_ip_address = true
  + availability_zone         = (known after apply)
  + disable_api_stop          = (known after apply)
  + disable_api_termination   = (known after apply)
  + ebs_optimized              = (known after apply)
  + enable_primary_ipv6       = (known after apply)
  + force_destroy              = false
  + get_password_data         = false
  + host_id                   = (known after apply)
  + host_resource_group_arn    = (known after apply)
  + iam_instance_profile       = (known after apply)
  + id                        = (known after apply)
  + instance_initiated_shutdown_behavior = (known after apply)
  + instance_lifecycle         = (known after apply)
  + instance_state             = (known after apply)
  + instance_type              = "t2.micro"
  + ipv6_address_count         = (known after apply)
  + ipv6_addresses             = (known after apply)
  + key_name                   = (known after apply)
  + monitoring                 = (known after apply)
  + outpost_arn                = (known after apply)
  + password_data              = (known after apply)
  + placement_group            = (known after apply)
  + placement_group_id         = (known after apply)
  + placement_partition_number = (known after apply)
  + primary_network_interface_id = (known after apply)
  + private_dns                = (known after apply)
  + private_ip                 = (known after apply)
  + public_dns                 = (known after apply)
  + public_ip                  = (known after apply)
  + region                     = "us-east-1"
  + secondary_private_ips      = (known after apply)
  + security_groups             = (known after apply)
  + source_dest_check          = true

```

Abbildung 4: Terraform Apply

2 Workshop 2: Advanced Networking

In the second workshop, we move away from the default VPC and build a custom network infrastructure. This includes creating a VPC, subnets, routing tables, and an Internet Gateway. We also assign a persistent Elastic IP to our web server.

2.1 Network Infrastructure

2.1.1 VPC and Internet Gateway

We create a new VPC with the CIDR block `10.0.0.0/16`. To enable communication with the internet, we attach an Internet Gateway to this VPC.


```
resource "aws_vpc" "web_vpc" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}
```

vpc-01642c1407e32f458 / tf-web-vpc Aktionen ▾

Details Info		Öffentlichen Zugriff blockieren	DNS-Hostnamen
VPC-ID vpc-01642c1407e32f458	Status Available	Aus	Aktiviert
DNS-Auflösung Aktiviert	Tenancy default	DHCP-Optionssatz dopt-0804897584b01e432	Haupt-Routing-Tabelle rtb-07e4fa61f19867c7f
Haupt-Netzwerk-ACL acl-0f155e51d51550947	Standard-VPC Nein	IPv4-CIDR 10.0.0.0/16	IPv6-Pool -
IPv6 CIDR (Network Border Group) -	Nutzungsmetriken für Netzwerkadressen Deaktiviert	Regelgruppen für Route-53-Resolver-DNS-Firewall Fehler beim Laden der Regelgruppen	Besitzer-ID 767397790708
Verschlüsselungssteuerungs-ID -	Verschlüsselungssteuerungsmodus -		

Ressourcenzuordnung Info CIDRs Flow-Protokolle Tags Integrationen

Ressourcenzuordnung Info Alle Details anzeigen

VPC
Ihr virtuelles AWS-Netzwerk
tf-web-vpc

Subnetze (1)
Subnetze innerhalb dieser VPC
us-east-1a
tf-web-subnet

Routing-Tabellen (2)
Netzwerkdatenverkehr an Ressourcen weiterleiten
tf-web-rt
rtb-07e4fa61f19867c7f

Abbildung 5: Custom VPC Created

igw-0c40f497ee5149ed6 / tf-web-igw Aktionen ▾

Details Info		VPC-ID	Eigentümer
Internet-Gateway-ID igw-0c40f497ee5149ed6	Status Attached	vpc-01642c1407e32f458 tf-web-vpc	767397790708

Tags (1) Tags verwalten

Tags suchen

Schlüssel	Wert
Name	tf-web-igw

Abbildung 6: Internet Gateway Attached

2.1.2 Subnet and Routing

We create a subnet (10.0.1.0/24) within our VPC. To allow traffic from the subnet to reach the internet, we create a custom route table that routes all traffic (0.0.0.0/0) to the Internet Gateway, and associate it with our subnet.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_subnet" "web_subnet" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = false
  tags = { Name = "tf-web-subnet" }
}

resource "aws_route_table_association" "web_rta" {
  subnet_id      = aws_subnet.web_subnet.id
  route_table_id = aws_route_table.web_rt.id
}
```

subnet-003febe3900aa2e0f / tf-web-subnet

Aktionen

Details

Subnetz-ID

subnet-003febe3900aa2e0f

IPv4-CIDR

10.0.1.0/24

Verfügbare Zone

use1-az1 (us-east-1a)

Netzwerk-ACL

acl-0f155e51d51550947

Kundeneigene IPv4-Adresse automatisch zuweisen

Nein

IPv6 CIDR-Reservierungen

-

Ressourcenname-DNS-A-AAAA-Datensatz

Deaktiviert

Subnetz-ARN

arn:aws:ec2:us-east-1:767397790708:subnet/subnet-003febe3900aa2e0f

Verfügbare IPv4-Adressen

250

Network Border Group

us-east-1

Standard-Subnetz

Nein

Kundeneigener IPv4-Pool

-

Nur IPv6

Nein

DNS64

Deaktiviert

Status

Available

IPv6-CIDR

-

VPC

vpc-01642c1407e32f458 | tf-web-vpc

Öffentliche IPv4-Adresse automatisch zuweisen

Nein

Stützpunkt-ID

-

Hostname-Typ

IP-Name

Eigentümer

767397790708

Öffentlichen Zugriff blockieren

Aus

IPv6-CIDR-Zuordnungs-ID

-

Routing-Tabelle

rtb-09d58ee8e0b88609c | tf-web-rt

IPv6-Adresse automatisch zuweisen

Nein

IPv4 CIDR-Reservierungen

-

Ressourcenname-DNS-A-Datensatz

Deaktiviert

Flow-Protokolle

Routing-Tabelle

Netzwerk-ACL

CIDR-Reservierungen

Freigeben

Tags

Routing-Tabelle: rtb-09d58ee8e0b88609c / tf-web-rt

Routing-Tabellenzuordnung bearbeiten

Routen (2)

Routen filtern

< 1 > ⚙

Bestimmungsort	Ziel
10.0.0.0/16	local
0.0.0.0/0	igw-0c40f497ee5149ed6

Abbildung 7: Subnet and Route Table Association

2.2 Compute and Access

2.2.1 Security Group

Similar to Workshop 1, we create a security group to allow HTTP traffic.

sg-06bad6ac684893f17 - tf-web-sg

Aktionen

Details

Name der Sicherheitsgruppe

tf-web-sg

Besitzer

767397790708

Sicherheitsgruppen-ID

sg-06bad6ac684893f17

Anzahl der Regeln für eingehenden Datenverkehr

2 Berechtigungseingaben

Beschreibung

Allow HTTP in, all egress

VPC-ID

vpc-01642c1407e32f458

Anzahl der Regeln für ausgehenden Datenverkehr

2 Berechtigungseingaben

Regeln für eingehenden Datenverkehr

Regeln für ausgehenden Datenverkehr

Wird freigegeben ... - neu

VPC-Zuordnungen - neu

Tags

Regeln für eingehenden Datenverkehr (2)

Tags verwalten

Regeln für eingehenden Datenverkehr bearbeiten

Suchen

< 1 > ⚙

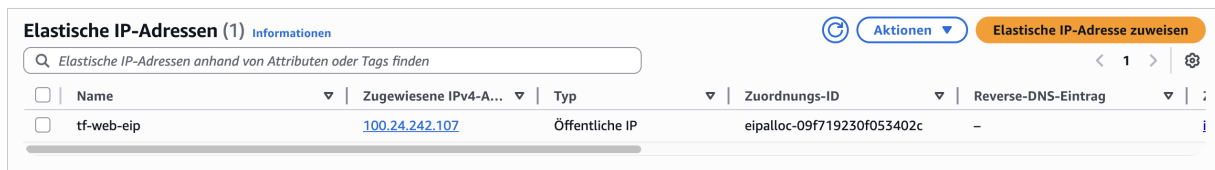
Abbildung 8: Security Group Inbound Rules

2.2.2 Network Interface and Elastic IP

Instead of relying on auto-assigned public IPs, we create a Network Interface (ENI) with a fixed private IP (10.0.1.50) and associate an Elastic IP (EIP) with it. This gives our server a static public IP address.

```
resource "aws_network_interface" "web_eni" {
  subnet_id      = aws_subnet.web_subnet.id
  private_ips    = ["10.0.1.50"]
  security_groups = [aws_security_group.web_sg.id]
  tags = { Name = "tf-web-eni" }
}

resource "aws_eip" "web_eip" {
  domain                = "vpc"
  network_interface     = aws_network_interface.web_eni.id
  associate_with_private_ip = "10.0.1.50"
  depends_on            = [aws_internet_gateway.igw]
  tags = { Name = "tf-web-eip" }
}
```



The screenshot shows the AWS Elastic IP console. At the top, it says 'Elastische IP-Adressen (1)' with an 'Informationen' link. Below this is a search bar and a table of allocated Elastic IP addresses. The table has columns for Name, Zugewiesene IPv4-Adresse, Typ, Zuordnungs-ID, and Reverse-DNS-Eintrag. One entry is shown: 'tf-web-eip' with the IP address '100.24.242.107', type 'Öffentliche IP', and Zuordnungs-ID 'eipalloc-09f719230f053402c'.

Name	Zugewiesene IPv4-Adresse	Typ	Zuordnungs-ID	Reverse-DNS-Eintrag
tf-web-eip	100.24.242.107	Öffentliche IP	eipalloc-09f719230f053402c	-

Abbildung 9: Elastic IP Allocation

2.2.3 EC2 Instance

We launch the EC2 instance and attach the previously created network interface. This ensures the instance uses our specific network configuration.

```
resource "aws_instance" "web" {
  ami          = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  network_interface {
    network_interface_id = aws_network_interface.web_eni.id
    device_index         = 0
  }

  user_data = <<-EOF
    #!/bin/bash
    sudo apt-get update
```

```

sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_World</h1>" | sudo tee /var/www/html/index.html
EOF

tags = { Name = "tf-apache-instance" }
}

```

Instance-Zusammenfassung für i-0b44d8e0decc8c1f3 (tf-apache-instance) [Info](#) [Verbinden](#) [Instance-Status](#) [Aktionen](#)

Vor less than a minute aktualisiert

Instance-ID i-0b44d8e0decc8c1f3	Öffentliche IPv4-Adresse 100.24.242.107 offene Adresse	Private IPv4-Adressen 10.0.1.50
IPv6-Adresse -	Instance-Status Läuft	Öffentlicher DNS ec2-100-24-242-107.compute-1.amazonaws.com offene Adresse
Hostname-Typ IP-Name: ip-10-0-1-50.ec2.internal	Private IP DNS-Name (nur IPv4) ip-10-0-1-50.ec2.internal	Elastic IP-Adressen 100.24.242.107 (tf-web-eip) [Öffentliche IP-Adresse]
DNS-Name der Antwort für private Ressourcen -	Instance-Typ t2.micro	AWS Compute Optimizer-Treffer Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. Weitere Informationen
Automatisch zugewiesene IP-Adresse -	VPC-ID vpc-01642c1407e32f458 (tf-web-vpc)	Auto-Scaling-Gruppenname -
IAM-Rolle -	Subnetz-ID subnet-003febe3900aa2e0f (tf-web-subnet)	Verwaltet Falsch
IMDSv2 Optional ⚠ EC2 empfiehlt, IMDSv2 auf erforderlichlich zu setzen. Weitere Informationen	Instance-ARN arn:aws:ec2:us-east-1:767397790708:instance/i-0b44d8e0decc8c1f3	
Operator -		

Abbildung 10: EC2 Instance Summary

2.3 Execution

We apply the configuration and verify the creation of resources. The output confirms the public IP of our instance.

```

> terraform fmt
> terraform validate
Success! The configuration is valid.

> terraform apply -auto-approve
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-00b13f11600160c10]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
+ create

Terraform will perform the following actions:

# aws_eip.web_eip will be created
+ resource "aws_eip" "web_eip" {
+   allocation_id      = (known after apply)
+   arn                = (known after apply)
+   associate_with_private_ip = "10.0.1.50"
+   association_id     = (known after apply)
+   carrier_ip        = (known after apply)
+   customer_owned_ip  = (known after apply)
+   domain             = "vpc"
+   id                 = (known after apply)
+   instance           = (known after apply)
+   ipam_pool_id       = (known after apply)
+   network_border_group = (known after apply)
+   network_interface  = (known after apply)
+   private_dns        = (known after apply)
+   private_ip         = (known after apply)
+   ptr_record         = (known after apply)
+   public_dns         = (known after apply)
+   public_ip          = (known after apply)
+   public_ipv4_pool    = (known after apply)
+   region             = "us-east-1"
+   tags               = {
+     "Name" = "tf-web-eip"
+   }
+   tags_all = {
+     "Name" = "tf-web-eip"
+   }
}

```

Abbildung 11: Terraform Apply Execution

3 Workshop 3: Load Balancing and High Availability

In this workshop, we implement a highly available web application architecture. We distribute traffic across two EC2 instances in different Availability Zones (AZs) using an Application Load Balancer (ALB). This ensures that if one AZ fails, the application remains accessible.

3.1 Networking Setup

3.1.1 VPC and Internet Gateway

We create a VPC and an Internet Gateway, similar to the previous workshop, to provide a network environment and internet connectivity.

```

resource "aws_vpc" "web_vpc" {
  cidr_block = "10.0.0.0/16"
}

```

```

enable_dns_support    = true
enable_dns_hostnames = true
tags = { Name = "tf-web-vpc" }
}

resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.web_vpc.id
  tags = { Name = "tf-web-igw" }
}

```

vpc-0d3e53f5fe370f9c8 / tf-web-vpc

Details | Ressourcenzuordnung | CIDRs | Flow-Protokolle | Tags | Integrationen

Details			
VPC-ID vpc-0d3e53f5fe370f9c8	Status Available	Öffentlichen Zugriff blockieren Aus	DNS-Hostnamen Aktiviert
DNS-Auflösung Aktiviert	Tenancy default	DHCP-Optionssatz dopt-0804897584b01e432	Haupt-Routing-Tabelle rtb-0c9d70a1e6b2db87c
Haupt-Netzwerk-ACL acl-014c8797788a6f9ab	Standard-VPC Nein	IPv4-CIDR 10.0.0.0/16	IPv6-Pool -
IPv6 CIDR (Network Border Group) -	Nutzungsmetriken für Netzwerkadressen Deaktiviert	Regelgruppen für Route-53-Resolver-DNS-Firewall Fehler beim Laden der Regelgruppen	Besitzer-ID 767397790708
Verschlüsselungssteuerungs-ID -	Verschlüsselungssteuerungsmodus -		

Abbildung 12: VPC Configuration

igw-0d6421b724ef21c6a / tf-web-igw

Details | Tags

Details			
Internet-Gateway-ID igw-0d6421b724ef21c6a	Status Attached	VPC-ID vpc-0d3e53f5fe370f9c8 tf-web-vpc	Eigentümer 767397790708

Abbildung 13: Internet Gateway

3.1.2 Subnets in Multiple AZs

To achieve high availability, we create two subnets in different Availability Zones (us-east-1a and us-east-1b).

```

resource "aws_subnet" "subnet_1" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "us-east-1a"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-1" }
}

```

```

}

resource "aws_subnet" "subnet_2" {
  vpc_id            = aws_vpc.web_vpc.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "us-east-1b"
  map_public_ip_on_launch = true
  tags = { Name = "tf-subnet-2" }
}

```

subnet-06da062a016089c28 / tf-subnet-1

Details			
Subnetz-ID subnet-06da062a016089c28	Subnetz-ARN arn:aws:ec2:us-east-1:767397790708:subnet/subnet-06da062a016089c28	Status Available	Öffentlichen Zugriff blockieren Aus
IPv4-CIDR 10.0.1.0/24	Verfügbare IPv4-Adressen 249	IPv6-CIDR -	IPv6-CIDR-Zuordnungs-ID -
Verfügbare Zone use1-az1 (us-east-1a)	Network Border Group us-east-1	VPC vpc-0d3e53f5fe370f9c8 tf-web-vpc	Routing-Tabelle rtb-071cf06657359c5ab tf-web-rt
Netzwerk-ACL acl-014c8797788a6f9ab	Standard-Subnetz Nein	Öffentliche IPv4-Adresse automatisch zuweisen Ja	IPv6-Adresse automatisch zuweisen Nein
Kundeneigene IPv4-Adresse automatisch zuweisen Nein	Kundeneigener IPv4-Pool -	Stützpunkt-ID -	IPv4 CIDR-Reservierungen -
IPv6 CIDR-Reservierungen -	Nur IPv6 Nein	Hostname-Typ IP-Name	Ressourcenname-DNS-A-Datensatz Deaktiviert
Ressourcenname-DNS-AAAA-Datensatz Deaktiviert	DNS64 Deaktiviert	Eigentümer 767397790708	

Abbildung 14: Subnet 1 in us-east-1a

subnet-0804b2f9df7fcbc67 / tf-subnet-2

Details			
Subnetz-ID subnet-0804b2f9df7fcbc67	Subnetz-ARN arn:aws:ec2:us-east-1:767397790708:subnet/subnet-0804b2f9df7fcbc67	Status Available	Öffentlichen Zugriff blockieren Aus
IPv4-CIDR 10.0.2.0/24	Verfügbare IPv4-Adressen 249	IPv6-CIDR -	IPv6-CIDR-Zuordnungs-ID -
Verfügbare Zone use1-az2 (us-east-1b)	Network Border Group us-east-1	VPC vpc-0d3e53f5fe370f9c8 tf-web-vpc	Routing-Tabelle rtb-071cf06657359c5ab tf-web-rt
Netzwerk-ACL acl-014c8797788a6f9ab	Standard-Subnetz Nein	Öffentliche IPv4-Adresse automatisch zuweisen Ja	IPv6-Adresse automatisch zuweisen Nein
Kundeneigene IPv4-Adresse automatisch zuweisen Nein	Kundeneigener IPv4-Pool -	Stützpunkt-ID -	IPv4 CIDR-Reservierungen -
IPv6 CIDR-Reservierungen -	Nur IPv6 Nein	Hostname-Typ IP-Name	Ressourcenname-DNS-A-Datensatz Deaktiviert
Ressourcenname-DNS-AAAA-Datensatz Deaktiviert	DNS64 Deaktiviert	Eigentümer 767397790708	

Abbildung 15: Subnet 2 in us-east-1b

3.1.3 Routing

We configure a route table to send internet traffic to the Internet Gateway and associate it with both subnets.

```
resource "aws_route_table" "web_rt" {
  vpc_id = aws_vpc.web_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "tf-web-rt" }
}

resource "aws_route_table_association" "rta_1" {
  subnet_id      = aws_subnet.subnet_1.id
  route_table_id = aws_route_table.web_rt.id
}

resource "aws_route_table_association" "rta_2" {
  subnet_id      = aws_subnet.subnet_2.id
  route_table_id = aws_route_table.web_rt.id
}
```

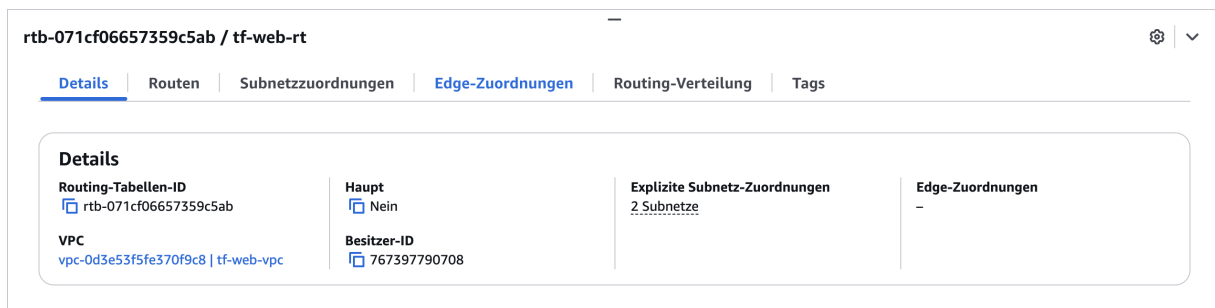


Abbildung 16: Route Table



Abbildung 17: Route Table Association

3.2 Security Groups

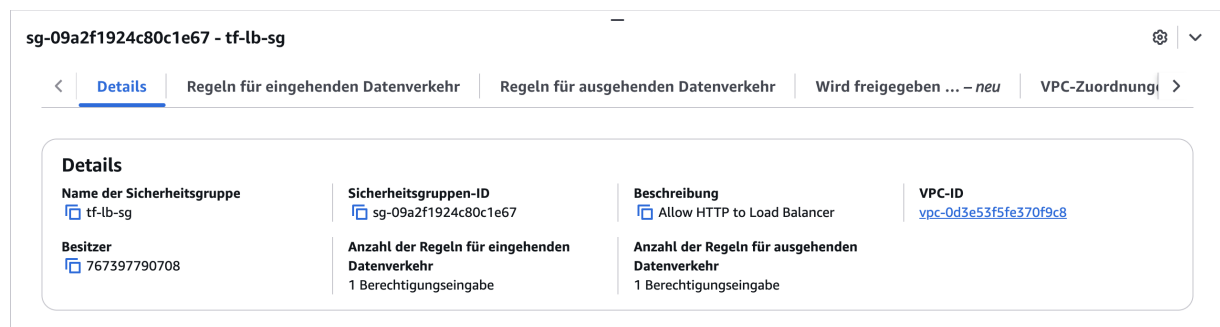
3.2.1 Load Balancer Security Group

This security group allows incoming HTTP traffic from anywhere to the Load Balancer.

```
resource "aws_security_group" "lb_sg" {
  name          = "tf-lb-sg"
  description   = "Allow_HTTP_to_Load_Balancer"
  vpc_id        = aws_vpc.web_vpc.id

  ingress {
    description = "HTTP_from_Internet"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```



The screenshot shows the AWS Management Console interface for a security group named 'sg-09a2f1924c80c1e67 - tf-lb-sg'. The 'Details' tab is selected, displaying a table with the following information:

Details			
Name der Sicherheitsgruppe tf-lb-sg	Sicherheitsgruppen-ID sg-09a2f1924c80c1e67	Beschreibung Allow HTTP to Load Balancer	VPC-ID vpc-0d3e53f5fe370f9c8
Besitzer 767397790708	Anzahl der Regeln für eingehenden Datenverkehr 1 Berechtigungseingabe	Anzahl der Regeln für ausgehenden Datenverkehr 1 Berechtigungseingabe	

Abbildung 18: Load Balancer Security Group

3.2.2 EC2 Security Group

The EC2 instances should only accept traffic from the Load Balancer. We reference the Load Balancer's security group ID in the ingress rule.

```
resource "aws_security_group" "ec2_sg" {
  name = "tf-ec2-sg"
```

```

description = "Allow_HTTP_from_LB"
vpc_id      = aws_vpc.web_vpc.id

ingress {
  description      = "HTTP_from_Load_Balancer"
  from_port        = 80
  to_port          = 80
  protocol         = "tcp"
  security_groups  = [aws_security_group.lb_sg.id]
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

```

sg-03f1a7278cc31611e - tf-ec2-sg

< **Details** | Regeln für eingehenden Datenverkehr | Regeln für ausgehenden Datenverkehr | Wird freigegeben ... - neu | VPC-Zuordnung >

Details			
Name der Sicherheitsgruppe tf-ec2-sg	Sicherheitsgruppen-ID sg-03f1a7278cc31611e	Beschreibung Allow HTTP from LB	VPC-ID vpc-0d3e53f5fe370f9c8
Besitzer 767397790708	Anzahl der Regeln für eingehenden Datenverkehr 1 Berechtigungseingabe	Anzahl der Regeln für ausgehenden Datenverkehr 1 Berechtigungseingabe	

Abbildung 19: EC2 Security Group restricted to LB

3.3 Load Balancer Configuration

3.3.1 Application Load Balancer

We create an external Application Load Balancer spanning our two subnets.

```

resource "aws_lb" "app_lb" {
  name                = "tf-web-lb"
  internal            = false
  load_balancer_type = "application"
  security_groups     = [aws_security_group.lb_sg.id]
  subnets            = [aws_subnet.subnet_1.id, aws_subnet.subnet_2.id]
  tags = { Name = "tf-web-lb" }
}

```

Load Balancer: tf-web-lb

<

Details

Listener und Regeln

Netzwerkzuordnung

Ressourcenkarte

Sicherheit

Überwachung

Integrationen

Attribute

>

Details

Load-Balancer-Typ

Anwendung

Status

✓

Aktiv

VPC

[vpc-0d3e53f5fe370f9c8](#)

IP-Adresstyp des Load Balancers

IPv4

Schema

Internet-facing

Gehostete Zone

Z35SXDOTRQ7X7K

Availability Zones

[subnet-0804b2f9df7fcbc67](#) us-east-1b (use1-az2)
 [subnet-06da062a016089c28](#) us-east-1a (use1-az1)

Datum erstellt

2. Dezember 2025, 22:31 (UTC+01:00)

Load-Balancer-ARN

[arn:aws:elasticloadbalancing:us-east-1:767397790708:loadbalancer/app/tf-web-lb/40af0919d6641b14](#)

DNS-Name

[tf-web-lb-504647856.us-east-1.elb.amazonaws.com](#) (A-Datensatz)

Abbildung 20: Application Load Balancer

3.3.2 Target Group and Listener

The target group defines where to route requests. The listener listens on port 80 and forwards traffic to this target group.

```
resource "aws_lb_target_group" "web_tg" {
  name      = "tf-web-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = aws_vpc.web_vpc.id

  health_check {
    path                = "/"
    interval             = 30
    timeout              = 5
    healthy_threshold    = 2
    unhealthy_threshold  = 2
  }
}

resource "aws_lb_listener" "front_end" {
  load_balancer_arn = aws_lb.app_lb.arn
  port              = "80"
  protocol          = "HTTP"

  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

Zielgruppe: tf-web-tg

Details | Ziele | Überwachung | Zustandsprüfungen | Attribute | Tags

Details

arn:aws:elasticloadbalancing:us-east-1:767397790708:targetgroup/tf-web-tg/c689c8a88b048a01

Zieltyp Instance	Protokoll : Port HTTP: 80	Protokollversion HTTP1	VPC vpc-0d3e53f5fe370f9c8
IP-Adresstyp IPv4	Load Balancer tf-web-lb		

2 Ziele insgesamt	2 Fehlerfrei 0 Anomal	0 Fehlerhaft	0 Ungenutzt	0 Anfänglich	0 Auslassen
----------------------	-----------------------------	-----------------	----------------	-----------------	----------------

► **Verteilung der Ziele nach Availability Zone (AZ)**
Wählen Sie Werte in dieser Tabelle aus, um die entsprechenden Filter anzuzeigen, die auf die unten angegebene Tabelle registrierter Ziele angewendet werden.

Abbildung 21: Target Group

Load Balancer: tf-web-lb

< Details | **Listener und Regeln** | Netzwerkzuordnung | Ressourcenkarte | Sicherheit | Überwachung | Integrationen | Attribute >

Listener und Regeln (1) Info

Ein Listener prüft auf Verbindungsanfragen an seinem konfigurierten Protokoll und Port. Der vom Listener empfangene Datenverkehr wird gemäß der Standardaktion und etwaigen zusätzlichen Regeln weitergeleitet.

Listener filtern

☐	Protokoll: Port	Standardaktion	Regeln	ARN	Sicherheitsrichtlinie	Standard-SSL-/TLS
☐	HTTP:80	Zur Zielgruppe weiterleiten tf-web-tg: 1 (100%) Klebrigkeit der Zielgruppe: Aus	1 Regel	ARN	Nicht zutreffend	Nicht zutreffend

Abbildung 22: Listener Configuration

3.4 Compute Resources

3.4.1 EC2 Instances

We launch two EC2 instances. We use the `count` parameter to create multiple instances and the modulo operator to alternate them between the two subnets. The user data script is customized to display the instance number.

```
resource "aws_instance" "web" {
  count          = 2
  ami            = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  subnet_id     = count.index % 2 == 0 ? aws_subnet.subnet_1.id : aws_subnet
    .subnet_2.id
  vpc_security_group_ids = [aws_security_group.ec2_sg.id]
```

```

user_data = <<-EOF
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl start apache2
sudo systemctl enable apache2
echo "<h1>Hello_from_Instance_${count.index}_1</h1>" | sudo tee /var/
    www/html/index.html
EOF

tags = { Name = "tf-web-instance-${count.index}_1" }
}

```

i-0342188f5d898d4ee (tf-web-instance-1)

Details

Status und Alarme

Überwachung

Sicherheit

Netzwerk

Speicher

Tags

▼ Instance-Zusammenfassung Info

Instance-ID

i-0342188f5d898d4ee

IPv6-Adresse

-

Hostname-Typ

IP-Name: ip-10-0-1-72.ec2.internal

DNS-Name der Antwort für private Ressourcen

-

Automatisch zugewiesene IP-Adresse

44.214.104.205 [Öffentliche IP-Adresse]

Öffentliche IPv4-Adresse

44.214.104.205 | offene Adresse

Instance-Status

Läuft

Private IP DNS-Name (nur IPv4)

ip-10-0-1-72.ec2.internal

Instance-Typ

t2.micro

VPC-ID

vpc-0d3e53f5fe370f9c8 (tf-web-vpc)

Private IPv4-Adressen

10.0.1.72

Öffentlicher DNS

ec2-44-214-104-205.compute-1.amazonaws.com | offene Adresse

Elastic IP-Adressen

-

AWS Compute Optimizer-Treffer

Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. | Weitere Informationen

Abbildung 23: EC2 Instance 1

i-01e10c1d7f5113f4c (tf-web-instance-2)

Details

Status und Alarme

Überwachung

Sicherheit

Netzwerk

Speicher

Tags

▼ Instance-Zusammenfassung Info

Instance-ID

i-01e10c1d7f5113f4c

IPv6-Adresse

-

Hostname-Typ

IP-Name: ip-10-0-2-153.ec2.internal

DNS-Name der Antwort für private Ressourcen

-

Automatisch zugewiesene IP-Adresse

98.93.191.240 [Öffentliche IP-Adresse]

Öffentliche IPv4-Adresse

98.93.191.240 | offene Adresse

Instance-Status

Läuft

Private IP DNS-Name (nur IPv4)

ip-10-0-2-153.ec2.internal

Instance-Typ

t2.micro

VPC-ID

vpc-0d3e53f5fe370f9c8 (tf-web-vpc)

Private IPv4-Adressen

10.0.2.153

Öffentlicher DNS

ec2-98-93-191-240.compute-1.amazonaws.com | offene Adresse

Elastic IP-Adressen

-

AWS Compute Optimizer-Treffer

Melden Sie sich für AWS Compute Optimizer an, um Empfehlungen zu erhalten. | Weitere Informationen

Abbildung 24: EC2 Instance 2

3.4.2 Target Group Attachment

We attach the created instances to the target group so the load balancer can route traffic to them.

```
resource "aws_lb_target_group_attachment" "web_attach" {
  count          = 2
  target_group_arn = aws_lb_target_group.web_tg.arn
  target_id      = aws_instance.web[count.index].id
  port          = 80
}
```

The screenshot shows the AWS Management Console interface for a target group named 'tf-web-tg'. The 'Ziele' (Targets) tab is selected, showing a list of registered targets. The table has columns for Instance-ID, Name, Port, Domäne, Integritätsstatus, Details zum Zustand, Admini..., and Übers... (likely Übersichten). Two targets are listed, both with a 'Healthy' status.

Instance-ID	Name	Port	Domäne	Integritäts...	Details zum Zustand	Admini...	Übers...
i-01e10c1d7f5113f4c	tf-web-instanc...	80	us-east-1b (us...	Healthy	-	No override.	No ove...
i-0342188f5d898d4ee	tf-web-instanc...	80	us-east-1a (us...	Healthy	-	No override.	No ove...

Abbildung 25: Target Group Attachment

3.5 Verification

After applying the configuration, we access the application via the Load Balancer's DNS name.

```
output "load_balancer_dns" {
  description = "The_DNS_name_of_the_load_balancer"
  value       = aws_lb.app_lb.dns_name
}
```

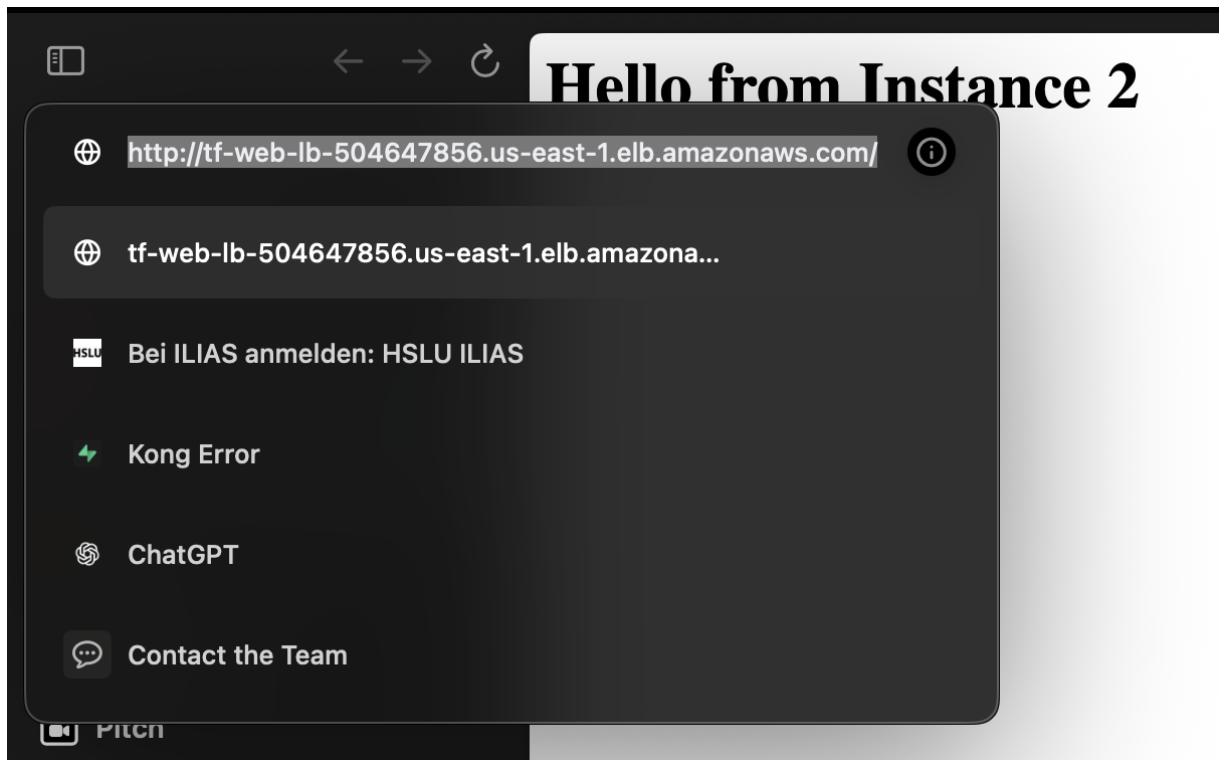


Abbildung 26: Website Accessed via Load Balancer

3.6 Automation and CI/CD

To automate the deployment process, we integrate GitHub Actions with Terraform Cloud. This allows us to automatically provision and manage our infrastructure upon code changes.

3.6.1 GitHub Configuration

We configure the necessary secrets in our GitHub repository to authenticate with Terraform Cloud.

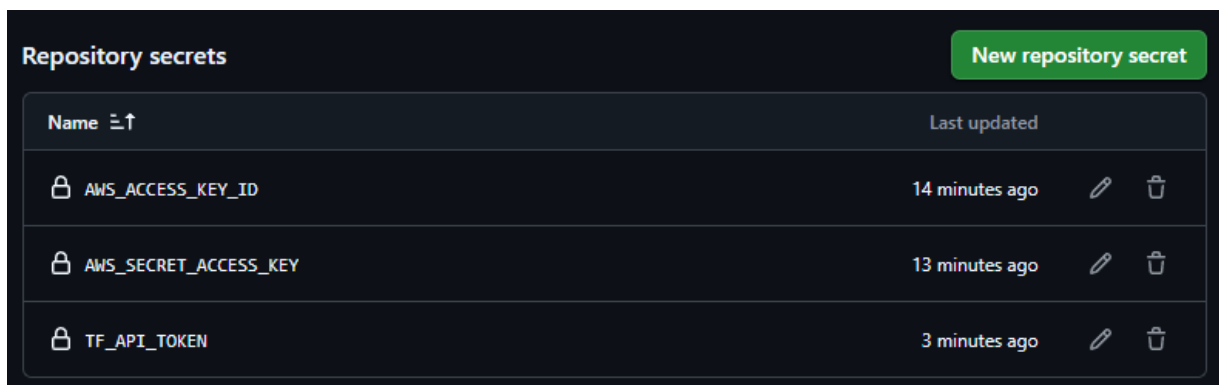


Abbildung 27: GitHub Repository Secrets

3.6.2 Terraform Cloud Configuration

In Terraform Cloud, we define the variables required for our infrastructure, such as AWS credentials and region settings.

Workspace variables (3)

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#).

Key	Value	Category	Actions
AWS_ACCESS_KEY_ID	ASIA3FLDZAP2NU3F6LLY	env	...
AWS_SECRET_ACCESS_KEY	C02gmFSowFD7fesiEHM3usYd8KmJ92Sqht oeGosc	env	...
AWS_SESSION_TOKEN	IQoJb3JpZ2luX2VjEFeaCXVzLXdlc3QtMiJlM EYCIQD32A57GWmbeCvDmH71p1eCnli25 e4E8L+8H41eZfo89glhAMWbPrKRO+dFlc4 JTOeTcWarw8RWAcNoIPA13b5IO74UKqYC CBoQABoMNzY3Mzk3NzkwNzA4lgysp0V0 +91C2S6TAVYqgwlpB00+grqHu3U1IZLa6K CPb0mEGbiYENUU40q/864udjJPFMkSnhf W1IVFQ0Neh4h3u2JtVkyqdQNdmKDFmze +PltxRg3S3r9cH6kUOw1YuKCOEzntBjFwdS UUIgkBc/oSKp/HQC7QzHox7XgZCHm8Jtn 7CatNdNXdUesYZvE+XA6OUIAsZ3n5FoW6 pSqcWunal66SBU6HP1njK9UljFYeDC3HsF1 83waztkRJdyhqERZriwu1vDJeZIKgW4o3iR/ Nfo+5UNs86+ZT2wmd+0s3lkAfaQyxUb/z gkC0Rmtu1kkY3hRwRc1GEILFDk11bke/Wf ouZ9tmKBXNKkKbABU8M0K8Mle8vMkGO pwBwGXtYkoNeST2RixMNmiXpP7NXdH8j7 KU/dVA6GdEGbXkTSOqvGo5+WBittVq49M MoZCHLmhT8Y6FPt+wTtp/pm3QILTU1kKE XhdvXy2Rdlyb/X5ZcsRekPQtBN2mp4oEWS RTCKwLyRCK4ztMfLYw4Ej/dvUZ+qrx9Lbn9 UybUjKtYfbbpMGWwhhIkqUW1ez2c2SRtheU qtU0oXXGn	env	...

Abbildung 28: Terraform Cloud Variables

3.6.3 CI/CD Pipeline

When code is pushed to the repository, a GitHub Action is triggered. This action communicates with Terraform Cloud to plan and apply the changes.

Deploy Infrastructure

Deploy Infrastructure #6

Summary

Jobs

terraform-apply

Run details

Usage

Workflow file

terraform-apply

successful run in 4m-45s

Search logs

Set up job

Checkout Code

Setup Terraform

terraform init

1 ▶ Run terraform init

2 ./aws/runner/terraform/init/terraform-init-aws-ec2-1234567890/terraform-init-aws-ec2-1234567890

3

4

5 Initializing Terraform Cloud...

6

7

8 Initializing provider plugins...

9 ▶ Finding backend run version matching "v1.0.0"...

10 ▶ Installing hashicorp/aws v4.20.0...

11 ▶ Installing hashicorp/aws v4.20.0 (signed by HashiCorp)

12

13 Terraform has created a lock file .terraform.lock.hcl to record the provider

14 selections it made above. Include this file in your version control repository

15 so that Terraform can guarantee to make the same selections by default when

16 you run "terraform init" in the future.

17

18 Terraform Cloud has been successfully initialized!

19

20 You may now begin working with Terraform Cloud. Try running "terraform plan" to

21 set any changes that are required for your Infrastructure.

22

23 If you ever set or change modules or Terraform Settings, run "terraform init"

24 again to reinitialize your working directory.

Terraform Apply

1 ▶ Run terraform apply -auto-approve

2 ./aws/runner/terraform/init/terraform-init-aws-ec2-1234567890/terraform-init-aws-ec2-1234567890

3

4

5 Running apply in Terraform Cloud. Output will stream here, pressing Ctrl-C

6 will cancel the remote apply if it's still pending. If the apply started to

7 will stop streaming the logs, but will not stop the apply running remotely.

8 Preparing the remote apply...

9

10 To view this run in a browser, visit:

11 https://cloud.terraform.io/runner/terraform-init-aws-ec2-1234567890

12

13 Waiting for the plan to start...

14 Terraform v1.0.0

15 on linux_amd64

16

17 Initializing plugins and modules...

18

19 data.aws_elb.aws_elb_frontend: Refresh module after 0s [id=aws-elb-frontend]

20

21 Terraform used the selected providers to generate the following execution

22 plan. Resource actions are indicated with the following symbols:

23 ▶ create

24

25 Terraform will perform the following actions:

26

27 # aws_instance[0] will be created

28

29 resource "aws_instance" "web" {

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429</

The execution of the plan can be monitored in Terraform Cloud.


infrc_workshop

infrc_workshop / Workspaces / workshop-3 / Runs / run-chukDgwEa2veo1dT

Force unlock
New run

Workspaces
workshop-3
Overview
Runs
States
Search & Import
Variables
Settings

workshop-3

ID: ws-n7oyNzK8qkApVhZ3

Add workspace description.

Running Resources 17 Tags 0 Terraform v1.14.0

Updated a few seconds ago

Triggered via CLI

Plan & apply duration: 4 minutes

Resources changed: +16 ~0 -0 Actions: 0 invoked

swipswup triggered a run from CLI 16 minutes ago
Run Details

Plan finished
16 minutes ago
Resources: 16 to add, 0 to change, 0 to destroy

Started 16 minutes ago > Finished 16 minutes ago

+ 16 to create

Filter resources by address...
Filter by operation
Show data sources Terraform 1.14.0
Download raw log

- aws aws_instance.web[0]
- aws aws_instance.web[1]
- aws aws_internet_gateway.igw
- aws aws_lb_listener.front_end
- aws aws_lb_target_group_attachment.web_attach[0]
- aws aws_lb_target_group_attachment.web_attach[1]
- aws aws_lb_target_group.web_tg
- aws aws_lb.app_lb
- aws aws_route_table_association.rta_1
- aws aws_route_table_association.rta_2
- aws aws_route_table.web_rt
- aws aws_security_group.ec2_sg
- aws aws_security_group.lb_sg
- aws aws_subnet.subnet_1
- aws aws_subnet.subnet_2
- aws aws_vpc.web_vpc

Outputs 1 planned to change

Download Sentinel mocks
Sentinel mocks can be used for testing your Sentinel policies

Apply finished
12 minutes ago
Resources: 16 added, 0 changed, 0 destroyed

Comment
Leave feedback or record a decision.
Add comment

Support Terms Privacy Security Accessibility © 2025 HashiCorp

Abbildung 30: Terraform Cloud Apply Run

25

Finally, we can also manage the destruction of resources through the pipeline or Terraform Cloud directly.



27